

Virtual Memory

Small TLBs

CS232 Operating Systems Fall 2019

Taj Muhammad Khan

TLBs are big

- Linear structures
- 32-bit address space with 4KB pages with an entry of 4 bytes
- Each process would require 4MB
- Imagine hundreds of processes running

One solution

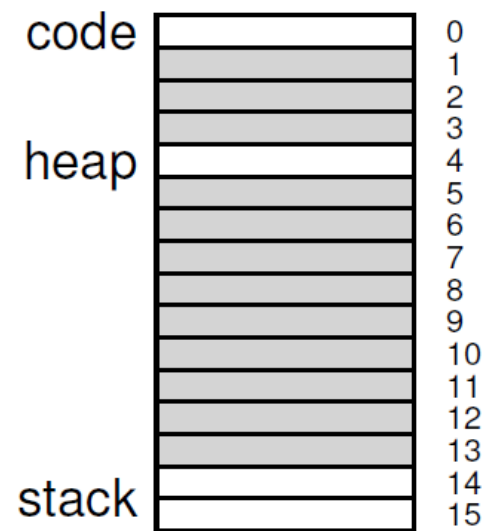
- Bigger pages
- Instead of 4KB page, have a 16KB page
- Pro: Will reduce the page table size 4 times
- Con: more internal fragmentation

2nd solution: Hybrid approach

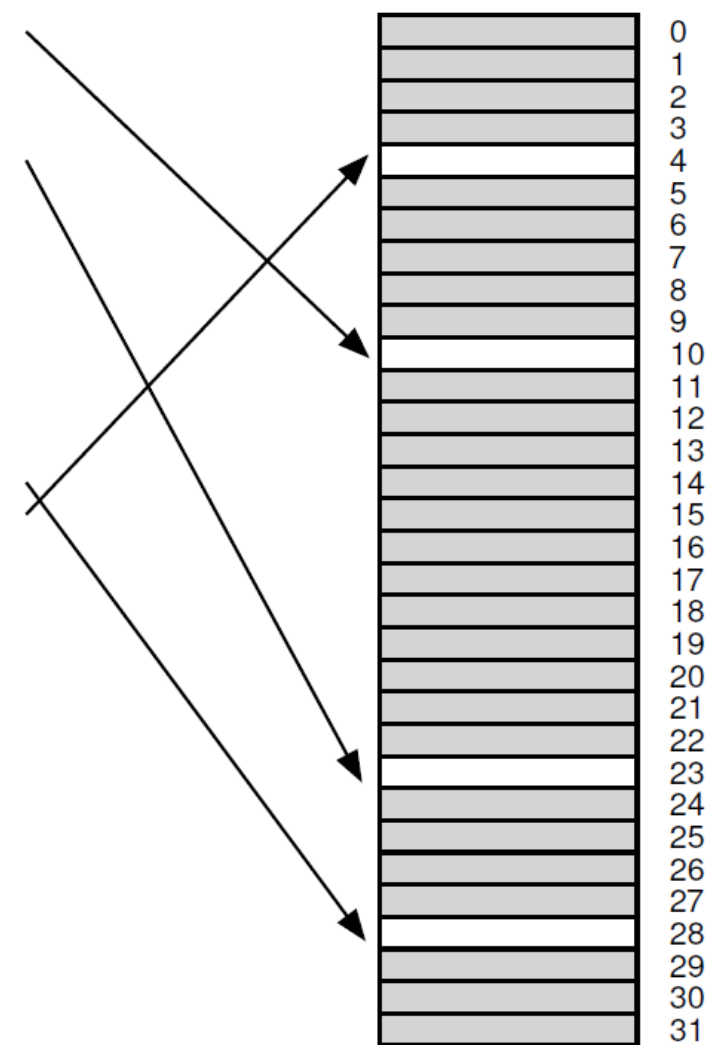
- Combines segmentation with paging
- We know most of the address space is unused
- Imagine a 16KB address space with small segments

PFN	valid	prot	present	dirty
10	1	r-x	1	0
-	0	—	-	-
-	0	—	-	-
-	0	—	-	-
23	1	rw-	1	1
-	0	—	-	-
-	0	—	-	-
-	0	—	-	-
-	0	—	-	-
-	0	—	-	-
-	0	—	-	-
-	0	—	-	-
-	0	—	-	-
-	0	—	-	-
28	1	rw-	1	1
4	1	rw-	1	1

Virtual Address Space



Physical Memory

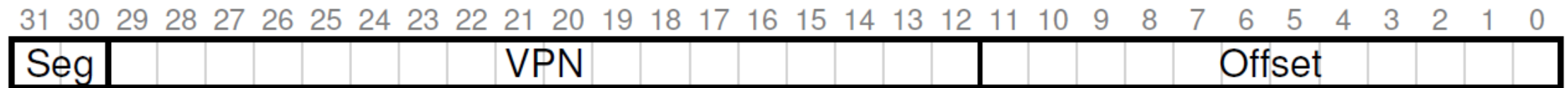


2nd solution: Hybrid approach

- Key insight:
 - Instead of having a page table per process,
 - Have one *per segment*
- Base and bound registers are used but:
 - Base points to the start of segment's page table
 - Bound points to the end of segment's page table

Example: Hybrid approach

- 32 bit address space, 4 KB pages
- Address space split into 4 parts (explicit approach)
- Each segment placed in a separate address space part



- Assume 3 segments, 3 base/bound pairs

- On TLB miss:

```
SN          = (VirtualAddress & SEG_MASK) >> SN_SHIFT
VPN         = (VirtualAddress & VPN_MASK) >> VPN_SHIFT
AddressOfPTE = Base[SN] + (VPN * sizeof(PTE))
```

2nd solution: Hybrid approach

- Pros:
 - Reduces page table sizes
- Cons:
 - Have multiple base and bound registers
 - Dividing address space into fixed size portions is not flexible
 - Variable sized page tables

3rd solution: Multi-level page tables

- Chop up the *page table* into page sized fragments
- If an entire page of page-table entries is invalid, don't allocate space for that page in page table RAM
- A new structure, page directory, tells you:
 - which *pages of the page table* are valid!
 - where in RAM would you find the valid pages of the page table

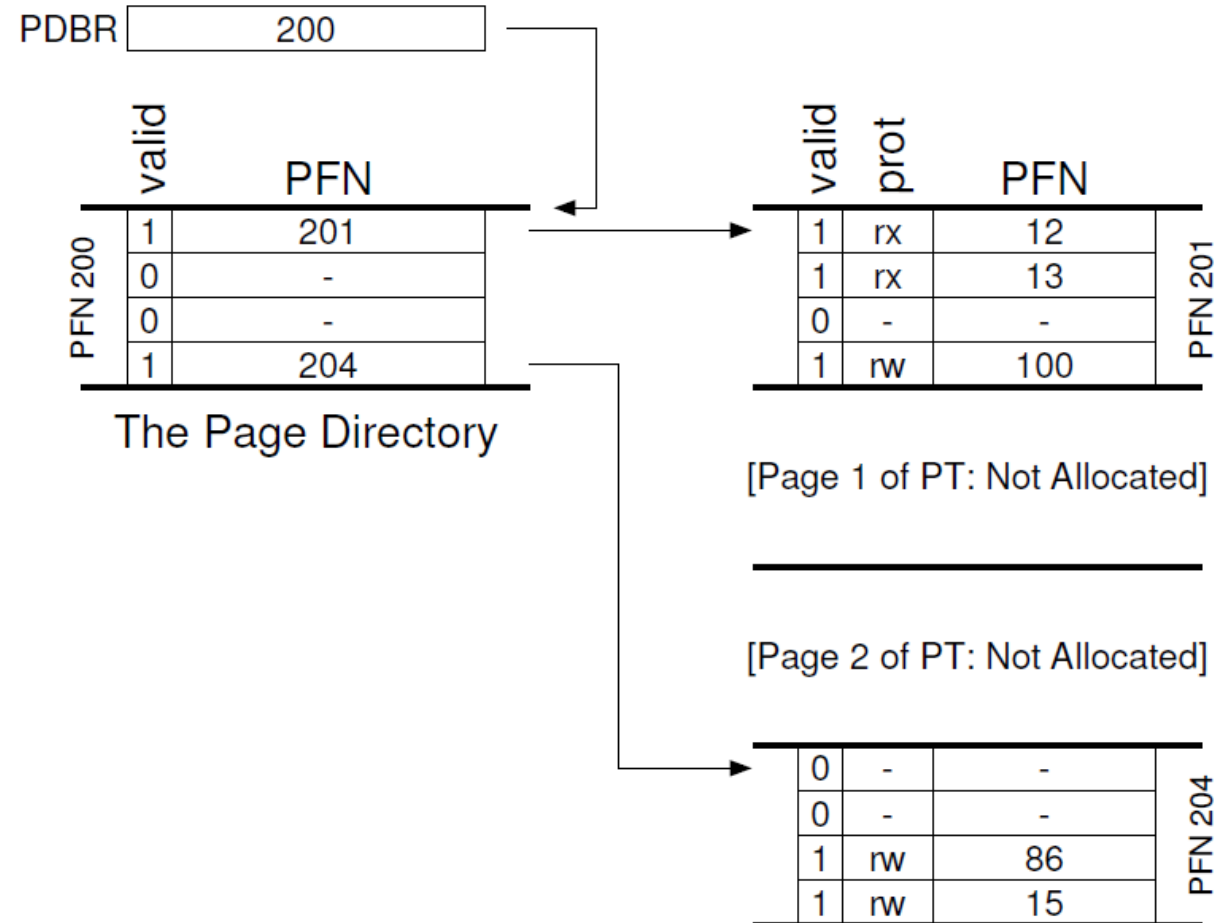
3rd solution: multi-level page tables

Linear Page Table

PTBR 201

valid	prot	PFN	
1	rx	12	PFN 201
1	rx	13	
0	-	-	
1	rw	100	
0	-	-	PFN 202
0	-	-	
0	-	-	
0	-	-	
0	-	-	PFN 203
0	-	-	
0	-	-	
0	-	-	
0	-	-	PFN 204
0	-	-	
0	-	-	
1	rw	86	
1	rw	15	

Multi-level Page Table



Multi-level page tables

- Advantages

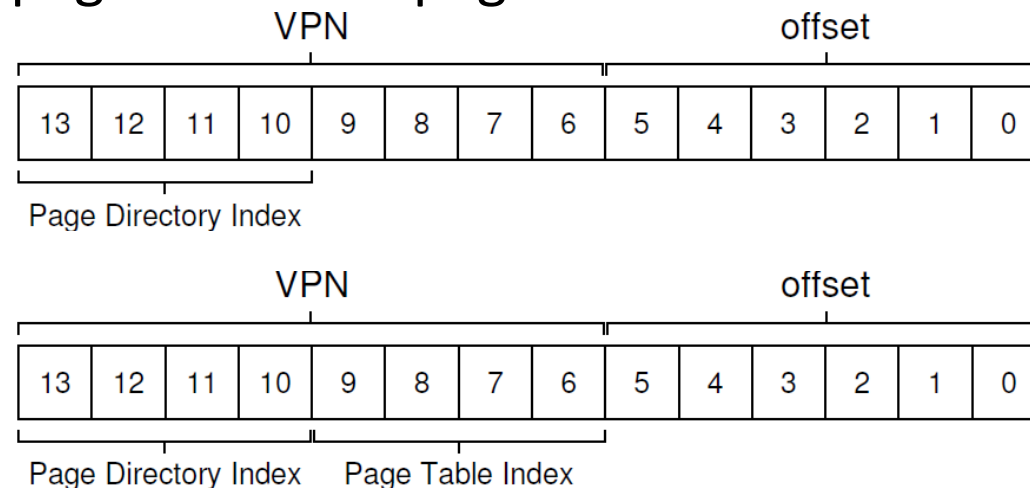
- Page table sizes are reduced
- Page tables can be distributed in non contiguous pages
 - Previous schemes required contiguous series of pages in memory

- Disadvantage

- On a TLB miss, we've to access the RAM ...
 - Three times !!!
 - example of Time-space tradeoff
- Complexity of implementation

Multi-level page tables – example

- 16KB address space, 64B pages
- Linear page table size?
- Multi-level page table:
 - Divide the page table into pages



0000 0000	code
0000 0001	code
0000 0010	(free)
0000 0011	(free)
0000 0100	heap
0000 0101	heap
0000 0110	(free)
0000 0111	(free)
.....	... all free ...
1111 1100	(free)
1111 1101	(free)
1111 1110	stack
1111 1111	stack

Multi-level page table – example

Page Directory		Page of PT (@PFN:100)			Page of PT (@PFN:101)		
PFN	valid?	PFN	valid	prot	PFN	valid	prot
100	1	10	1	r-x	—	0	—
—	0	23	1	r-x	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	80	1	rw-	—	0	—
—	0	59	1	rw-	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	55	1	rw-
101	1	—	0	—	45	1	rw-

Figure 20.5: A Page Directory, And Pieces Of Page Table

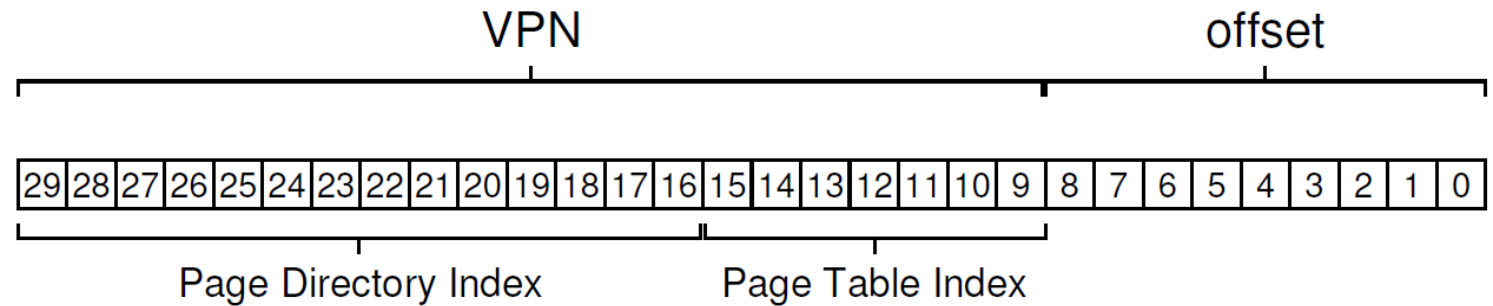
Multi-level p

```
1  VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2  (Success, TlbEntry) = TLB_Lookup(VPN)
3  if (Success == True)    // TLB Hit
4      if (CanAccess(TlbEntry.ProtectBits) == True)
5          Offset    = VirtualAddress & OFFSET_MASK
6          PhysAddr  = (TlbEntry.PFN << SHIFT) | Offset
7          Register = AccessMemory(PhysAddr)
8      else
9          RaiseException(PROTECTION_FAULT)
10 else    // TLB Miss
11     // first, get page directory entry
12     PDIndex = (VPN & PD_MASK) >> PD_SHIFT
13     PDEAddr = PDBR + (PDIndex * sizeof(PDE))
14     PDE      = AccessMemory(PDEAddr)
15     if (PDE.Valid == False)
16         RaiseException(SEGMENTATION_FAULT)
17     else
18         // PDE is valid: now fetch PTE from page table
19         PTIndex = (VPN & PT_MASK) >> PT_SHIFT
20         PTEAddr = (PDE.PFN << SHIFT) + (PTIndex * sizeof(PTE))
21         PTE      = AccessMemory(PTEAddr)
22         if (PTE.Valid == False)
23             RaiseException(SEGMENTATION_FAULT)
24         else if (CanAccess(PTE.ProtectBits) == False)
25             RaiseException(PROTECTION_FAULT)
26         else
27             TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
28             RetryInstruction()
```

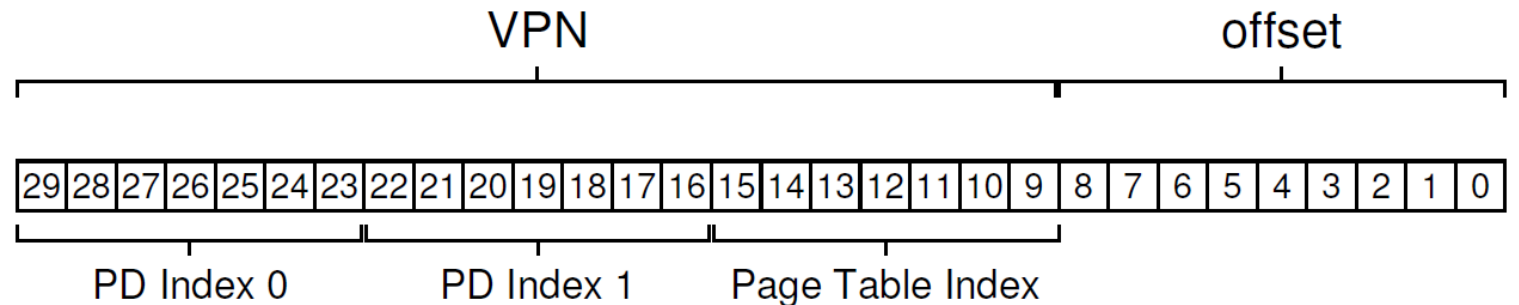
Figure 20.6: Multi-level Page Table Control Flow

Multi-level page tables – more than 2 levels

- 30 bit address space, 512 byte pages



- Page directory doesn't fit in one page



Solution 4: Inverted page tables

- One page table for the whole system
- Indices are PFNs instead of VPNs
- PTE contains: PID, VPN
- Have to do a linear search
 - Hash tables?

Next chapter

- What if the size of total number of pages in a system exceeds RAM?
- What if there are so many page tables that the RAM space allocated to kernel falls short?